

Compiler-Known Value Quantization (CKVQ)

Role-Aware KV Cache Quantization with Compile-Time Pre-RoPE Insertion, Page-Granular Scaling, and Sensitivity-Weighted Bit Allocation in NeuralScript

Authors: Brandon Wiemer Date: April 2026 Status: Research Proposal

Abstract

We propose **CKVQ (Compiler-Known Value Quantization)**, a KV cache quantization technique for NSL that exploits NSL's compile-time visibility of the attention layer graph, RoPE application point, Pre-Attention Paged-Cache Allocation (PCA) page boundaries, and GEBA (Gradient-Envelope Bit Allocation) symbolic bounds to produce KV quantization decisions that external frameworks cannot match. Where KVQuant (NeurIPS 2024) requires calibration data and Python-level instrumentation to achieve pre-RoPE per-channel key quantization, and where KIVI (ICML 2024) and TurboQuant (ICLR 2026) use runtime algorithms with no compile-time input, CKVQ derives per-layer bit budgets symbolically from model structure, inserts pre-RoPE quantization automatically (no user API change), and uses PCA pages as the natural granularity for scale factors. The result is a KV cache quantization scheme that: (1) supports 2-4 bit per value with near-zero accuracy loss; (2) enables 10M-token context on a single 8-GPU system (matching KVQuant); (3) requires no user-facing annotation (purely inferred from model structure); (4) composes with the NVFP4 KV cache format on Blackwell; (5) integrates with CFIP's format system. Expected impact: 4-8× KV cache memory reduction at under 0.5% accuracy loss; 2-4× higher concurrent request capacity on the same hardware; 5-10× context extension at fixed hardware.

1. The KV Cache Memory Wall

1.1 KV cache scaling

For a transformer with L layers, H attention heads, head dimension d_h , at sequence length S and batch size B , the KV cache stores:

$$\text{KV bytes} = 2 \cdot L \cdot H \cdot d_h \cdot S \cdot B \cdot \text{bytes_per_value}$$

Concrete numbers for modern models:

Model	Context	Batch	FP16 KV cache
Llama-3-8B	8K	1	4 GB
Llama-3-8B	128K	1	64 GB
Llama-3-70B	128K	1	40 GB (GQA)
Llama-3-70B	1M	1	328 GB
Llama-3-70B	128K	32	1280 GB

At 128K context, the KV cache alone exceeds the weight footprint of most models. At 1M context, the cache exceeds the capacity of any single GPU. For LLaMA-70B at 1M tokens, even with GQA reducing the count, the KV cache is approximately 328 GB — across four H100s just for cache.

Inference throughput at long context is memory-bandwidth bound during decode: every generated token reads the entire KV cache. A 328 GB cache read per token is the bottleneck. Reducing KV cache size directly improves throughput.

1.2 What existing techniques do

Technique	Bit budget	Accuracy	Notes
KVQuant (NeurIPS 2024)	2-4 bit	Near-lossless	Pre-RoPE, NUQ, dense-and-sparse, attention-sink-aware; 10M context on 8-GPU
KIVI (ICML 2024)	2 bit	<1% loss	Asymmetric, per-channel K, per-token V; 2.6× reduction
TurboQuant (ICLR 2026)	3.5 bit	Near-lossless	Training-free, PolarQuant + QJL; 6× reduction
NVFP4 KV cache (NVIDIA)	4 bit	<1% loss	Blackwell-native E2M1+FP8 scales; block-granular; hardware-accelerated
PagedAttention (vLLM)	FP16 (no quant)	None	Memory-allocation technique; 2-4× higher batch sizes via fragmentation elimination

Each addresses a piece. None integrate all of:

- Compile-time knowledge of which layers have RoPE, which use GQA, which are attention-sink-dominant.
- Symbolic per-layer bit-budget derivation (no calibration data needed to start).
- Page-granular scale factors (composing with paged allocation).
- Hardware-format awareness (NVFP4 on Blackwell vs FP8 on Hopper vs INT4 on earlier).

CKVQ aims at all five simultaneously.

1.3 Why NSL should have this

NSL has:

- **PCA** (paged KV cache) — allocation management in fixed-size blocks.
- **CFA** — compile-time-emitted attention kernels with known @rope annotation.
- **CFIP** — format system that could apply to KV cache.
- **GEBA** — symbolic bit-budget bound mechanism.

None of these target KV cache specifically. The integration is missing. CKVQ is the integration.

2. Six Compiler-Native Quantization Innovations

2.1 Innovation 1: Compile-time pre-RoPE quantization insertion

KVQuant's central insight: RoPE rotation scrambles the outlier structure of keys. Before RoPE, outliers are concentrated in a small set of channels; after RoPE, outliers spread across channels. Quantizing *before* RoPE preserves the per-channel outlier structure and enables much lower bit budgets (2-bit keys viable vs 4-bit post-RoPE).

In PyTorch, inserting pre-RoPE quantization requires manual instrumentation: override the forward pass of the attention layer, inject quantization between the KV projection and the RoPE application. Users must know to do this, and it breaks if the RoPE implementation changes.

In NSL, the RoPE application point is visible to the compiler. The `@rope` annotation on the attention layer tells CFA to emit RoPE after the KV projection; CKVQ inserts quantization at exactly that point. The user writes standard attention code with `@rope`; CKVQ does the right thing automatically.

The MIR transformation looks like:

```

# Before CKVQ
K_proj_out = matmul(X, W_k)           # [B, S, H*d_h]
K_rope = apply_rope(K_proj_out, freqs) # [B, S, H, d_h]
kv_cache_store(K_rope, position)       # full precision

# After CKVQ
K_proj_out = matmul(X, W_k)
K_quant, K_scale = quantize(K_proj_out, format=per_channel_2bit) # pre-RoPE!
kv_cache_store_quant(K_quant, K_scale, position)                  # compressed
storage
# At read time:
K_quant, K_scale = kv_cache_load_quant(position)
K_proj_dequant = dequantize(K_quant, K_scale)
K_rope = apply_rope(K_proj_dequant, freqs)

```

The dequantize-then-apply-RoPE path is slightly more expensive than applying RoPE to already-dense keys, but the memory bandwidth saving at read time dominates. KVQuant shows this tradeoff as strongly favoring the pre-RoPE approach.

2.2 Innovation 2: Symbolic per-layer bit budget via GEBA

GEBA's symbolic analysis, developed for CFIP, computes an upper bound on the quantization error each layer can tolerate without degrading loss by more than a declared epsilon. For interior tensors, GEBA assigns a bit budget per tensor role.

CKVQ extends GEBA to KV cache tensors. The extension is mechanical: KV cache values are attention-layer outputs with known downstream use (read once per subsequent decoding step, multiplied against queries and softmaxed). GEBA's existing bounds for accumulator tensors and linear-projection outputs cover keys and values respectively.

The per-layer bit assignment that results:

- **Layer 0 (near embedding):** less sensitive; bit budget typically 2 for keys, 2 for values.
- **Attention-sink layers (layer 0-2 in most models):** extra sensitivity for the first few tokens; bit budget higher (3-4) for the first-N positions, lower (2) for the rest. KVQuant's attention-sink-aware special-casing.
- **Middle layers:** moderate sensitivity; 2-3 bits typical.
- **Last layer (near LM head):** higher sensitivity; 3-4 bits typical.
- **GQA layers:** each group's K/V shared across multiple Q; more aggressive compression OK (KIVI observation).

No calibration data is required for these assignments — GEBA derives them symbolically from model architecture. Calibration, if available, refines the bounds (typically tightening them, permitting more aggressive compression) but is not the primary source of decisions.

2.3 Innovation 3: PCA page-granular scale factors

PCA allocates KV cache in pages (typically 16 or 32 tokens per page). Each page holds a contiguous range of the sequence for a given layer and head.

Natural quantization granularity:

- **Per-token scale:** fine-grained but adds $O(S)$ metadata bytes.
- **Per-tensor scale:** coarse; a single outlier token ruins the scale for the whole sequence.
- **Per-page scale:** Goldilocks — pages are small enough to bound outlier influence (one bad page doesn't ruin another), but large enough that metadata overhead is negligible (one scale per 16-32 tokens = $<1\%$ of bytes).

CKVQ stores one scale per page per format field. For keys in a 32-token page at `head_dim=128` with 2-bit quantization:

- Values: $32 \times 128 \times 2 \text{ bits} = 1024 \text{ bytes}$.
- Scale (FP16): 2 bytes.
- Overhead: 0.2%.

For comparison, KIVI's per-token scale has 2 bytes per token, or $32 \times 2 = 64$ bytes overhead per page — 6%. Per-page is an order of magnitude better metadata overhead at comparable quantization quality.

2.4 Innovation 4: Dense-and-sparse outlier handling per page

KVQuant's dense-and-sparse decomposition stores the top-k% outliers at full precision and the remaining 99%+ at low precision. For PCA pages, the natural implementation is a per-page outlier list:

- Each page has a fixed outlier budget (e.g., 2 outliers per 32-token page = $\sim 6\%$ of tokens).
- Outlier positions are stored as indices; outlier values are stored at FP16.
- Non-outlier values use the page's chosen low-bit format.
- Memory per outlier: 4 bytes (2 for index, 2 for value). At 2 outliers $\times$ 128 `head_dim` per page, outlier storage is 1 KB per page — comparable to the quantized value storage.

This handles the attention-sink token issue automatically: attention-sink tokens are always outliers in KV magnitude, so they land in the outlier list and are preserved at full precision.

2.5 Innovation 5: Hardware-format awareness

CKVQ selects the underlying representation based on target hardware:

- **Ampere / Hopper (SM_80, SM_90):** INT4 for values (mature INT4 dequant kernels), E4M3 FP8 for keys (M35 instructions available).
- **Blackwell (SM_100):** NVFP4 (E2M1 with FP8 E4M3 scales) — hardware-accelerated NVFP4 tensor core paths.
- **Earlier (SM_75 / Turing):** FP8 emulation or INT4 fallback; less aggressive.

The format choice is made at compile time via `--gpu-arch` and is baked into the emitted kernels. Cross-hardware deployment uses NSL's multi-target compilation.

Specifically, on Blackwell, the attention kernel uses NVFP4 MMA paths directly on the quantized KV values, avoiding the dequantize step entirely. This is a $2\times$ throughput improvement over dequant-then-MMA paths.

2.6 Innovation 6: Composition with CFIP format registry

CKVQ's formats are registered with CFIP's format system. An IF4 tensor (4-bit CFIP interior) and a CKVQ 4-bit KV page use the same underlying bit layout. This unifies the quantization infrastructure: one dequant kernel template serves both CFIP interior and CKVQ KV cache reads.

The difference between CFIP interior and CKVQ KV cache is *residency*: CFIP interior stays in SMEM/registers; CKVQ KV lives in HBM. CKVQ is an instance of the "semi-interior CFIP" concept from the cross-paper connections paper — a CFIP format legally used in HBM under strict producer/consumer contracts.

3. Format Specifications

3.1 CKVQ-2bit (aggressive, for insensitive layers)

Per-page layout:

- Non-outlier values: 2-bit integer per element, symmetric signed (values in $\{-2, -1, 0, 1\} \times \text{scale}$).
- Scale: FP16, one per page per head.
- Outliers: up to k per page (default $k = 2$), each as (index: u8, value: f16). Budget: $4k$ bytes per page.
- Metadata: 2 bytes (scale) + $4k$ bytes (outliers) + 1 byte (outlier count).

Compression ratio vs FP16: $\sim 8\times$ for 32-token pages at `head_dim=128`.

3.2 CKVQ-4bit (balanced, for typical layers)

Per-page layout:

- Non-outlier values: 4-bit integer per element, symmetric signed.
- Scale: FP16, one per page per head.
- Outliers: up to k per page (default $k = 1$).
- Metadata: 2 bytes (scale) + 4 bytes (outlier) + 1 byte (outlier count).

Compression ratio vs FP16: $\sim 4\times$.

3.3 CKVQ-NVFP4 (Blackwell, hardware-native)

Per-block layout (block size 16, matching NVFP4 spec):

- Values: E2M1 FP4 per element (4 bits per value).
- Scale: FP8 E4M3 per block (1 byte per 16 elements).
- No outlier list (NVFP4's FP4 format handles the range naturally).

Compression ratio vs FP16: $4\times$. Hardware-accelerated via Blackwell's NVFP4 tensor cores — no dequant needed in attention.

3.4 CKVQ-8bit (conservative, for attention-sink layers)

Per-page layout:

- Values: FP8 E4M3 (1 byte per value).
- Scale: FP16 per page per head.
- No outlier list needed.

Compression ratio vs FP16: $2\times$. Used when GEBA's bound does not admit the aggressive 2-bit or 4-bit formats.

3.5 Per-layer format selection

Layer position	Key format	Value format
Layer 0 (first few tokens)	CKVQ-8bit	CKVQ-8bit
Layer 0 (remaining tokens)	CKVQ-4bit	CKVQ-4bit
Middle layers	CKVQ-2bit	CKVQ-4bit (asymmetric, KIVI-style)
Last 1-2 layers	CKVQ-4bit	CKVQ-4bit

The asymmetric middle-layer choice (2-bit keys, 4-bit values) comes from KIVI's observation that keys tolerate lower bit budgets than values for typical LLMs.

4. Implementation Architecture

4.1 Pipeline position

CKVQ operates across Stages 4 (WGGO), 5 (MIR lowering), 6 (codegen), and 7 (runtime).

Stage 4 (WGGO):

1. Identify attention layers and their @rope/@gqa annotations.
2. Apply GEBA per-layer bit-budget derivation.
3. Assign per-layer key/value formats.
4. Write decisions to WGGO decision table.

Stage 5 (MIR lowering):

1. Insert pre-RoPE quantize op after KV projection.
2. Insert dequant op before RoPE in attention forward.
3. Adjust kv_cache_store/kv_cache_load to use per-page quantized layout.
4. Allocate per-page scale metadata in the page structure.

Stage 6 (codegen):

1. CFA's attention kernel emitter reads CKVQ format and emits appropriate PTX.
2. Quantize and dequantize kernels are emitted (fused with surrounding ops where possible).
3. On Blackwell, NVFP4-specific MMA paths are emitted for CKVQ-NVFP4 format.

Stage 7 (runtime):

1. PCA page allocation extended to include per-page scale and outlier metadata.
2. KV cache reads go through CKVQ dequant path.
3. Page eviction and migration unchanged (quantized data is opaque to the allocator).

4.2 PCA page structure extension

Current PCA page:

```
struct Page {  
    data: [seq_slot][head_dim][dtype],  
    metadata: { layer_id, head_group, sequence_id, token_range },  
}
```

CKVQ extended page:

```

struct CKVQPage {
    // Quantized value storage (compact layout)
    quantized: [seq_slot][head_dim][quantized_dtype],

    // Per-page scale factor
    scale: f16,

    // Outlier list (up to k outliers)
    outliers: [
        { position: u16, value: f16 };
        k
    ],
    outlier_count: u8,

    // Format identifier (compile-time constant per layer, but carried for dispatch)
    format: CKVQFormat,

    // Standard metadata
    metadata: { layer_id, head_group, sequence_id, token_range },
}

```

Memory overhead per page relative to a full-precision page: approximately equal to scale + outlier storage, typically <2% of compressed data size. Net compression factor is close to the nominal (e.g., 4× for CKVQ-4bit).

4.3 Attention kernel dequant path

The CFA attention kernel is extended to dequantize KV values on the fly. The dequant is fused into the K-loop prologue:

```

// Per-tile K-loop iteration
for k_block in k_blocks:
    // Load K values from HBM (quantized)
    K_quant = load_kv_page(k_block, layer_id, head_id)

    // Fused dequant (small math, hidden behind memory latency)
    K_float = dequantize_per_page(K_quant.data, K_quant.scale, K_quant.format)
    patch_outliers_inplace(K_float, K_quant.outliers, K_quant.outlier_count)

    // Apply RoPE (post-dequant)
    K_rope = apply_rope(K_float, freqs[k_block])

    // Standard attention: Q @ K^T, softmax, weighted V
    ...

```

The dequant cost is typically 10-20 cycles per element at CKVQ-4bit, with memory latency 200-500 cycles per cache line — so the dequant is fully hidden. Net effect: no measurable compute overhead, full bandwidth savings.

4.4 Quantize-on-store kernel

At KV cache store time (end of each forward pass for the current position):

```
// After KV projection, before RoPE
K_proj = matmul(X, W_k)    // [B, H, d_h]
V_proj = matmul(X, W_v)    // [B, H, d_h]

// Compute per-page scale (page may hold up to 32 new tokens being filled)
K_scale = compute_absmax(K_proj) / 2^(bit_budget - 1)

// Identify outliers (values exceeding threshold)
K_outliers = identify_outliers(K_proj, threshold = threshold_factor * K_scale)

// Quantize non-outliers
K_quant = quantize(K_proj \ K_outliers, K_scale, bit_budget)

// Store to page
store_to_page(page, K_quant, K_scale, K_outliers)
```

Outlier identification adds a small overhead (~1-2% of step time at typical outlier rates). It is performed only at store time, not at read time (outliers are stored as pre-identified; read-time path just patches them in).

5. Integration with NSL Pipeline

5.1 Decision ordering

CKVQ runs after CFA's decisions (which determine RoPE placement, GQA structure) and after GEBA's analysis (which provides per-layer bit budgets). It runs before MIR lowering.

5.2 New invariants

I-28 — CKVQ quantization is inserted pre-RoPE for keys. This is a CFA contract: the RoPE application happens after dequantization on read, or equivalently, quantization happens before RoPE on write.

I-29 — PCA pages are the granularity of CKVQ scales. A page has one scale per head; all tokens within a page share the scale. No intra-page scale variation.

I-30 — Outlier list capacity is compile-time bounded. Per-page outlier count is at most k (configurable, default $k = 2$ for 2-bit, $k = 1$ for 4-bit). If more than k outliers are detected during quantization, the page's scale is re-derived to absorb them (at the cost of range).

I-31 — CKVQ format changes trigger cache invalidation. Changing the per-layer format requires flushing the KV cache. This is only done at training-time phase transitions (CTQS) or at initial model load; never in-flight during inference.

5.3 Failure modes

F-18 — Outlier overflow. Symptom: some pages produce saturated values. Root cause: more than k outliers in a single page, and the adaptive scale is not wide enough. Mitigation: the quantize-on-store kernel falls back to a wider scale and logs a warning. Repeated occurrences suggest k should be increased for that layer.

F-19 — RoPE application on quantized tensor. Symptom: wrong results at dequantization. Root cause: a pass incorrectly fused RoPE before dequant (violating I-28). Mitigation: MIR verification pass ensures pre-RoPE quantization.

F-20 — Page-scale misalignment with paged allocation. Symptom: reading wrong scale from wrong page. Root cause: page metadata layout change without coordinator. Mitigation: PCA and CKVQ agree on the CKVQPage struct layout via a shared header. Compile-time ABI check.

6. Evaluation Plan

6.1 Baselines

1. **FP16 KV cache (no quant):** memory baseline.
2. **KIVI 2-bit:** runtime baseline.
3. **KVQuant 4-bit:** best-accuracy academic baseline.
4. **TurboQuant 3.5-bit:** training-free academic baseline.
5. **NVFP4 KV cache (Blackwell):** hardware-native baseline.
6. **CKVQ (this work):** across all configurations.

6.2 Benchmarks

- **Perplexity on wikitext-2** — standard quantization accuracy metric.
- **LongBench** — long-context accuracy across tasks.
- **Needle-in-a-Haystack** — retrieval accuracy at extreme context.
- **Throughput** — tokens/s at batch sizes 1, 4, 16, 64 across contexts 8K, 32K, 128K.
- **Memory** — KV cache footprint for fixed sequence.

6.3 Context-extension demonstration

Replicating KVQuant's demonstration:

- Llama-style 7B at 1M context on a single A100 80GB.
- Llama-style 7B at 10M context on 8× A100 (single node).
- NSLCoder-50M at 100K context on RTX 5070 Ti 16GB.

CKVQ should match or exceed KVQuant on these demonstrations while requiring no calibration data.

6.4 Correctness tests

1. **Numerical equivalence vs uncompressed** at declared tolerance for each format.
2. **Pre-RoPE quantization invariant** — MIR verification that quantize op precedes RoPE.
3. **Outlier preservation** — 100% of identified outliers preserved bitwise.
4. **Page-scale coverage** — every page's values are representable at the page's scale after quantization (no saturated values except identified outliers).
5. **NVFP4 path correctness on Blackwell** — MMA-on-quantized results match dequant-then-MMA to within floating-point rounding tolerance.

6.5 Implementation phases

Phase	Scope	Sessions
1	CKVQ analysis pass in WGGO	1
2	GEBA extension for KV cache	1
3	PCA page structure extension	1
4	Quantize-on-store kernels (all formats)	2
5	CFA attention-kernel dequant-on-read integration	2
6	Outlier list handling	1
7	NVFP4 path for Blackwell	1
8	Correctness tests	1
9	Benchmarks (KIVI, KVQuant, TurboQuant comparisons)	1
Total		11

Estimated scope: ~1900 LOC across five files, 11 implementation sessions.

7. Future Work

Dynamic quantization ratios per head (DMC-style). Dynamic Memory Compression (Nawrot et al., 2024) learns different compression ratios per head and layer via continued pre-training. CKVQ's per-layer budgets are static per compile; a learned version would require training-time adjustment, similar to CTQS's phase system.

Cross-layer key sharing (MLA-style). Multi-Latent Attention compresses KV to a latent representation shared across layers, reducing cache size by another factor. Integrating this into CKVQ requires MLA-specific MIR changes.

Sparse KV eviction composition. Techniques like H2O and SnapKV evict tokens that have low attention scores. CKVQ is complementary: eviction reduces token count, CKVQ reduces bytes per token. The composition is straightforward (CKVQ applies to retained tokens) but benchmarking the combination is future work.

8. Conclusion

The KV cache is the single largest consumer of inference memory at long context and the primary determinant of both throughput and context capacity. CKVQ provides NSL with a first-class KV cache quantization technique that leverages compile-time visibility of RoPE, GQA, attention-sink structure, and PCA page boundaries to match or exceed the accuracy/compression Pareto of every external technique while requiring no user-facing calibration or manual instrumentation. With CKVQ shipped, NSL's inference story extends from weight quantization (CFTP) and interior precision (CFIP) to KV cache quantization — the three main levers of LLM inference memory.

References

1. Hooper, C. et al. (2024). *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization*. NeurIPS 2024.
2. Liu, Z. et al. (2024). *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache*. ICML 2024.
3. Google Research. (2026). *TurboQuant: Fast Vector Quantization via Random Rotation and Residual Compression*. ICLR 2026.
4. NVIDIA. (2025). *Optimizing Inference for Long Context and Large Batch Sizes with NVFP4 KV Cache*. Technical Blog.
5. Kwon, W. et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023.
6. Nawrot, P. et al. (2024). *Dynamic Memory Compression: Retrofitting LLMs for Accelerated Inference*. arXiv:2403.09636.
7. Zhang, Z. et al. (2023). *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models*. NeurIPS 2023.